IBEX RISC-V · 68 SECURITY ASSERTIONS · 9 MODULES · TARGET IEEE ETS 2027

Assertion Quality Ranker

Three formal metrics, computed on the clean Ibex design before any Trojan exists, give each of the 68 security assertions a quality score. A 39-Trojan benchmark then tests whether that score predicts real detection.

Advisor approved: SRTLC · AER · SAPC

Comparison only: CWE/CVSS · Trojan category

Red/blue review: SAPC and AER definitions need fixing

FPV and SPV 1 · SRTLC 2 · AER 3 · SAPC AQS score Comparison tables Five stages Experiments Open issues

monitored / proven · unmonitored / counterexample · test signal · score

---

BACKGROUND

Two JasperGold engines, two kinds of blind spot

All 68 assertions are already proven on clean Ibex (BEC 2026 AutoTrans paper). The question here is how much each one protects. Two JasperGold apps answer different halves of that question.

Formal Property Verification (FPV)

```
Can any sequence of input values
break this property?
```

Explores every value on every cycle and returns either a proof or a counterexample trace. Used by **AER**.

Security Path Verification (SPV)

```
Which inputs have a path through
the logic to this signal?
```

Traces how the design is connected, without trying values. Used by **SAPC**.

> Bank-vault picture: FPV checks whether any code on the keypad opens the door. SPV checks whether an air duct leads into the vault.

1 · Security RTL Coverage (SRTLC)

JasperGold COI report · 0 new runs

Question: of the RTL that experts call security-relevant, how much does at least one assertion watch?

1. Every signal named in an NS31A expert property is **security-critical**.
2. Map those signals to Ibex. Unmapped signals still count as gaps.
3. List the RTL lines that drive each security-critical signal.
4. Read each assertion's cone of influence (COI): all logic that can affect it.
5. Count the lines that fall inside at least one COI.

$$\text{SRTLC} = \frac{\text{security lines inside } \cup \text{ COI}}{\text{all security lines in module}} \times 100\%$$

> **Per assertion:** SRTLC_contrib = share of security lines that only this assertion covers.

L101 L108 L115 L122 L129 L136 L143 L150 L157 L164
L171 L178 L185 L192 L199 L206 L213 L220 L227 L234
L241 L248 L255 L262 L269 L276 L283 L290 L297 L304
L311 L318 L325 L332 L339 L346 L353 L360 L367 L374

75% 30 of 40 security lines watched (worked example, CSR)

Each cell is one RTL line that drives an expert-flagged signal. Red cells are lines no assertion's cone of influence reaches. Dividing by all 2,000 module lines instead would hide them.

2 · Assertion Evasion Resistance (AER)

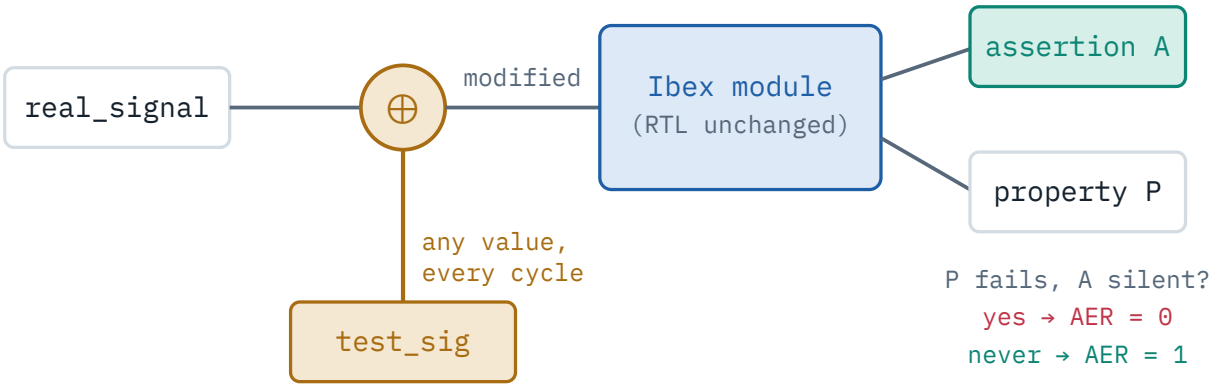
JasperGold FPV + test signal · 10 runs

Question: if a security-critical signal can be flipped at any time, in any pattern, can that cause a violation without the assertion noticing?

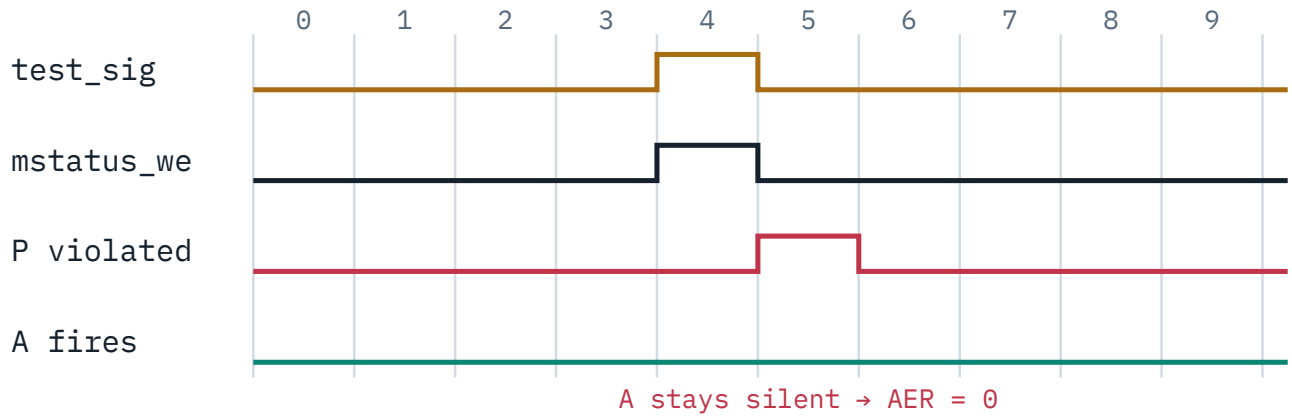
1. Pick a security-critical signal, e.g. `mstatus_we`.
2. Add an undriven input, the **test signal** `test_sig`.

3. Wire `modified = real ^ test_sig` in a wrapper; the RTL is unchanged.
4. FPV asks: does any `test_sig` sequence break security property P while assertion A stays silent?
5. Proven → **AER = 1**. Counterexample → **AER = 0** with the exact evasion trace.

> **Open issue:** P must be a separate reference property. If P is A itself, AER is always 1. See open issues.



Counterexample | Proven



Cycle 4: the test signal flips the write enable and the property is violated. Proven means A always fires in such cases, for every possible test-signal sequence.

3 · Security Assertion Path Coverage (SAPC)

`JasperGold SPV · 10 runs`

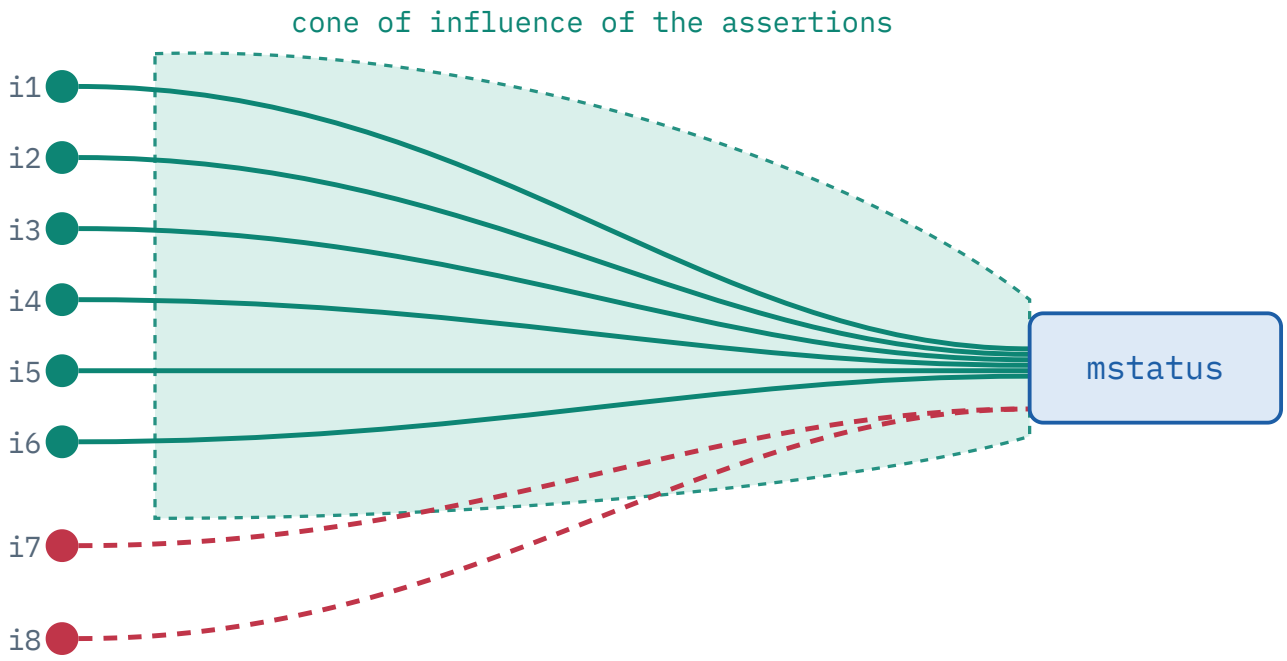
Question: is there a route into a security signal that no assertion watches at all?

1. For each security-critical signal, ask SPV for every path from a chip input.
2. Check whether each path lies inside some assertion's cone of influence.
3. Paths outside every cone are **structural blind spots**.

$$\text{SAPC} = \frac{\text{SPV paths inside some assertion COI}}{\text{all SPV paths into security signals}} \times 100\%$$

> **Why both AER and SAPC:** AER only tampers with the signal the test signal is attached to. Routes it never touched stay invisible to it.

> **Prior work:** Eslami, Ghasempouri, Pagliarini (ISQED 2022, arXiv 2201.01130) defined an SPV-based security coverage for assertions. SAPC extends our own group's metric; it is not the first SPV metric.



75% 6 of 8 paths monitored (worked example)

Each dot on the left is a primary input. Green paths are inside an assertion's cone. The two dashed red paths reach `mstatus` with nothing watching them.

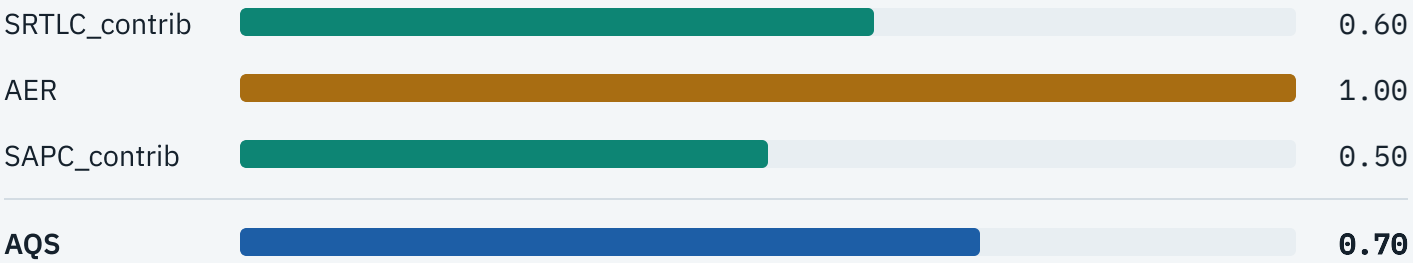
COMPOSITE

Assertion Quality Score (AQS)

```
AQS = (SRTLC_contrib + AER + SAPC_contrib) / 3
```

Each part lies between 0 and 1 and gets equal weight, giving one ranked score for every assertion.

The test of the score: run the same assertions on the 39-Trojan benchmark, count the Trojans each one detects, and check the correlation with AQS. A strong positive correlation is the paper's central claim: a formal score computed before any Trojan existed predicts real detection.



> Worked example for one assertion, not a measured result.

DESCRIPTIVE, NOT SCORED

Two comparison tables beside the score

A · Assertion → weakness

Each assertion is mapped to the CWE weakness it guards and that weakness's CVSS severity, e.g. PMP → CWE-1220. Grounded in NIST CSWP 45 (hardware threat and sensitivity from CWE/CAPEC).

B · Assertion → Trojan category

Each assertion is mapped to the TrustHub categories it addresses: Availability, Covert, Denial of Service, Integrity, Leak, Privilege. After the Trojan stage, the table also shows which categories it actually detected.

FLOW

Five stages, 98 new JasperGold runs

Stage 1 · AVS

Assertion Validation

Done in BEC 2026: 68 assertions proven and non-vacuous.

0 new runs

Stage 2 · AQRS

Quality Ranking

SRTLC, AER, SAPC on clean RTL, then AQS and both tables.

10 FPV + 10 SPV

Stage 3 · TES

Trojan Evaluation

Assertions on 39 trojaned designs; correlate AQS with detection.

39 FPV

Stage 4 · ARS

Refinement

Python templates add assertions for the gaps from stage 2. No LLM.

0 runs

Stage 5 · RAVS

Refined Validation

Re-run on the Trojans and measure the improvement.

39 FPV

WHAT TO RUN

Experiments and the figure each one produces

Core experiments are the agreed plan. Proposed experiments come from the red-team review and still need the advisor's approval.

ID	Experiment	Stage · tool	Runs	Output / figure	Status
E1	SRTLC per module and per assertion from the BEC COV reports	AQRS · Python	0	Bar chart per module; map of unwatched security lines	core

ID	Experiment	Stage · tool	Runs	Output / figure	Status
E2	AER with test-signal wrappers, one per bind file	AQRS · FPV	10	AER table; evasion traces for every AER = 0	core
E3	SAPC structural paths into each security signal	AQRS · SPV	10	Path coverage per module; list of blind-spot paths	core
E4	AQS ranking of all 68 assertions	AQRS · Python	0	Ranked table, top and bottom assertions per module	core
E5	CWE/CVSS and TrustHub-category comparison tables	AQRS · Python	0	Tables A and B	core
E6	Run unchanged assertions on the 39 trojaned designs	TES · FPV	39	Detection heat map (Trojans × assertions); TDER, WTDR, AAD	Trojans
E7	Correlate AQS with detection count per assertion	TES · Python	0	Scatter plot; Pearson and Spearman with confidence intervals	core
E8	Refine assertions from the gap report	ARS · Python	0	List of new template assertions per gap	core
E9	Re-run refined assertions on the Trojans	RAVS · FPV	39	Before/after TDER, WTDR, AAD	Trojans
E10	Baselines: does AQS beat cone size, assertion length or a random ranking?	TES · Python	0	Correlation of each baseline next to AQS	proposed
E11	Hold out part of the Trojans during refinement so E9 is not tested on the same Trojans that guided it	ARS/RAVS	0 extra	Improvement on held-out Trojans only	proposed

ID	Experiment	Stage · tool	Runs	Output / figure	Status
E12	Graded AER: smallest tampering budget (bits × cycles) that evades each assertion	AQRS · FPV	`0 extra`	AER as a score in [0,1] instead of 0/1; binary vs graded ablation	proposed

RED-TEAM AND BLUE-TEAM REVIEW · DECISIONS NEEDED

Open issues before the definitions are frozen

FATAL IF TRUE · SAPC

SAPC may repeat SRTLC

A cone of influence already contains every path feeding a signal. Any assertion that reads `mstatus` covers every path into it, so SAPC tends to be 100% or 0%.

Fix both teams agree on: count an SPV *functional* path (input → security signal → the assertion's failure output), not a syntactic cone. Present SAPC as a security-scoped extension of Eslami, Ghasempouri and Pagliarini (2022), whose node coverage was never tested against Trojans.

MAJOR · AER

Property P is undefined, and the mechanism is published

If P is the assertion itself, AER is always 1. The red team found the same XOR free-input model in Chuah et al. (RISC-V Summit Europe 2025 abstract); the blue team could not confirm this, so read the paper. Both teams relate AER to formal mutation coverage (Chockler et al., CAV 2001).

Fix: define P separately (the NS31A reference property or a golden check), make AER graded by the smallest tampering budget that evades the assertion (E12), and credit the prior models openly.

MAJOR · VALIDATION

The correlation test is fragile

Small sample (68 assertions, 39 Trojans), mostly 0/1 detection counts, Trojans written by the same group, and refinement re-tested on the same Trojans.

Fix: freeze the Trojans first, use Spearman with confidence intervals and control for module and

MAJOR · POSITIONING

No "first" claims survive

Per-assertion SPV ranking (Eslami 2022, SCARF 2024), assertion ranking (Ayalasomayajula HOST 2024), TDER/TPI on Ibex (our ISCAS 2025 paper) and RISC-V Trojan benchmarks already exist.

cone size, add baselines (E10) and a held-out split (E11). Write "TrustHub categories, new Ibex RTL", since TrustHub's own designs are contested.

Claim instead: the combination of FPV evasion and SPV path evidence in one per-assertion score, computed Trojan-free and shown to correlate with detection on an independent benchmark.

> Full reports in `docs/research/`: `red_team.md`, `blue_team.md`, `literature_survey.md` (72 papers), `new_metric_ideas.md`, and the summary `review_summary.md`.

Sharjeel Imtiaz, Uljana Reinsalu, Tara Ghasempouri · Tallinn University of Technology · Estonian Research Council PSG837 · Input assertions from *AutoTrans* (BEC 2026). Worked examples use illustrative numbers; no Stage 2–5 results exist yet.